CSE 21 Fall 2026
Homework 5
Submitted: 11/09/2026

David Washington(200484943), Judah Boyce(200453537),
Osinachi Agaranna(200455712), Nick Young(200455733)

Problem 1

(a) Trace of FunctionSolver($x^2 - x - 1$, 4, 0.0625)

Here $2e = 0.125$ and $f(v) = v^2 - v - 1$. The entries in the last three columns shows the updated values after the iteration.

Iteration	$hi - lo > 2e$	$f(v)$	Sign	lo	hi	v
0	—	—	—	0	4	2
1	True	1	$>$	0	2	1
2	True	-1	$<$	1	2	1.5
3	True	-0.25	$<$	1.5	2	1.75
4	True	0.3125	$>$	1.5	1.75	1.625
5	True	0.015625	$>$	1.5	1.625	1.5625
6	False	—	—	1.5	1.625	1.5625

Return: 1.5625.

(b) Loop invariant

Claim: After each iteration, $lo \leq r \leq hi$, where $f(r) = 0$.

Base case: Before the first iteration, the algorithm sets $lo = 0$ and $hi = n$. The problem states $0 < r < n$. Therefore $lo = 0 \leq r \leq n = hi$, so the invariant holds before the loop begins.

Inductive hypothesis: Suppose that after t iterations, $lo \leq r \leq hi$. We must show the invariant remains true after iteration $t + 1$.

Inductive Step: Set $v = (lo + hi)/2$. Since $lo \leq r \leq hi$, consider the following cases.

- If $f(v) = 0$, then $v = r$ and the algorithm returns the root.
- If $f(v) < 0$, because f is increasing and $f(r) = 0$, we have $v < r$. The algorithm sets $lo = v$, so $lo \leq r \leq hi$ remains true.
- If $f(v) > 0$, because f is increasing and $f(r) = 0$, we have $v > r$. The algorithm sets $hi = v$, so $lo \leq r \leq hi$ remains true.

In every case in which the algorithm continues, the root remains between lo and hi. Therefore, by induction, $lo \leq r \leq hi$ holds after every iteration.

Problem 2

QueueSort: Initially, $Q = ([a_1], [a_2], \ldots, [a_n])$. While $|Q| \geq 2$, dequeue the first two sorted lists, merge them, and enqueue the result.

(a) Prove the loop invariant using induction.

Invariant: After each iteration, every list in Q is sorted.

Base case: Initially,

$$Q = ([a_1], [a_2], \ldots, [a_n]).$$

Every list contains exactly one element. A single-element list is automatically sorted. Therefore, the invariant holds before the loop begins.

Inductive hypothesis: Suppose that after t iterations, every list in Q is sorted.

Inductive step: During iteration $t + 1$, the algorithm removes the first two lists from Q. By the inductive hypothesis, both lists are sorted. The MERGE subroutine combines there two lists into a sorted list. The resulting list is added to Q, while all remaining lists stay unchanged and sorted. Therefore, every list in Q remains sorted after iteration $t + 1$.

Thus, by induction, the loop invariant holds after each iteration.

(b) Use the loop invariant to show that the algorithm is correct.

Initially, Q contains n lists. Each iteration removes two lists and adds one decreasing the number of lists by one. Thus, after $n - 1$ iterations (for $n \geq 1$), exactly one list remains and the loop terminates.

By the loop invariant, that list is sorted. Because MERGE combines its input without adding or removing elements, the final lists contains exactly the original input elements. Therefore, QueueSort correctly sorts the input.

(c) Show an $O(n^2)$ bound using the product rule.

The while loop executes $n - 1 = O(n)$ times, since each iteration decreases the number of lists by one. Each iteration calls MERGE on two lists containing j and k elements. Because the lists together contain at most n elements, $j + k \leq n$ so each merge takes $O(j + k) = O(n)$ time. Queue operations contribute only constant overhead per iteration.

By the product rule,

$$T(n) = O(n) \cdot O(n) = \boxed{O(n^2)}.$$

(d) Show that the quadratic bound is not tight.

A more careful analysis consider the total work across group of merges, rather than treating every merge as size n. Because the algorithm dequeues from the front an enqueues each merged list at the back, lists are combined in rounds of roughly doubling size. The resulting merge tree is balanced: each original element participates in at most $\lceil \log_2 n \rceil$ merges.

Each merge of lists of lengths j and k costs $O(j + k)$, so we can charge a constant amount of work to each element participating in that merge. Since there are n elements and each is charged at most $O(\log n)$ times, the total merge work is

$$O\left(\sum_{i=1}^{n} \log n\right) = O(n \log n).$$

The $n-1$ queue iterations add $O(n)$ overhead. Hence,

$$T(n) = O(n \log n) + O(n) = \boxed{O(n \log n)}.$$

This is asymptotically smaller than $O(n^2)$, so the quadratic upper bound is not tight. Moreover, with linear time merging, the balanced merge tree gives $\Theta(n \log n)$ total time for $n \geq 2$.

Problem 3

(a)

The total number of prints is

$$\begin{aligned}\sum_{i=1}^{n}(n+i) &= \sum_{i=1}^{n} n + \sum_{i=1}^{n} i \\ &= n^2 + \frac{n(n+1)}{2} \\ &= \frac{2n^2 + n^2 + n}{2} \\ &= \boxed{\frac{3n^2+n}{2}}.\end{aligned}$$

Therefore, the total is $\Theta(n^2)$.

(b)

For each i, the inner loop runs $2n - 2i + 1$ times. Thus

$\sum_{i=1}^{n}(2n-2i+1)$
$=\sum_{i=1}^{n}(2n+1) - 2\sum_{i=1}^{n} i$

$=$n(2n+1)-2$\left(\frac{n(n+1)}{2}\right)$

$=2n^2 + n - n^2 - n$

$=\boxed{n^2}$.

Therefore, the total number of iterations is $\Theta(n^2)$.

(c)

The algorithm prints once for every choice of three distinct indices $1 \leq i < j < k \leq n$. Hence

$\mathrm{n}\binom{}{3}$
$=\mathrm{n}!\frac{}{3!(n-3)!}$
$=\mathrm{n(n\text{-}1)(n\text{-}2)}\frac{}{6}$
$=\boxed{\frac{n^3-3n^2+2n}{6}}$.

Therefore, the total number of prints is $\Theta(n^3)$.